

# 分割統治

アルゴリズムの世界で「難攻不落の大きな問題」をスマートに解くための強力な戦略が「**分割統治法 (Divide and Conquer)**」です。

クイックソートやマージソートの基礎となっている考え方で、基本情報技術者試験でも頻出の概念です。

## 1. 分割統治法とは？

言葉の通り、問題を「分割」して「統治（解決）」する手法です。具体的には以下の3ステップを繰り返します。

1. **分割 (Divide)**: 大きな問題を、同じ形式の「より小さな問題」に切り分ける。
2. **統治 (Conquer)**: 小さくなった問題をそれぞれ解決する。（十分に小さければそのまま解き、そうでなければさらに分割する）
3. **統合 (Combine)**: 小さな問題の解決結果を組み合わせ、最終的な答えを作る。

## 2. なぜ分割すると速くなるのか？

例えば「100万件のデータから特定の数字を探す」と、「50万件のデータから探す」のでは、後者の方が圧倒的に楽ですね。

分割統治法を適用すると、「**問題の規模が小さくなるスピード**」が劇的に上がるため、計算時間を大幅に短縮できます。多くの分割統治アルゴリズムは、計算量が に抑えられるのが大きなメリットです。

## 3. 分割統治法を採用している代表的なアルゴリズム

| カテゴリ | アルゴリズム名 | 分割のやり方                     |
|------|---------|----------------------------|
| 探索   | 二分探索    | 範囲を半分に絞り込み、探す範囲を狭めていく。     |
| ソート  | マージソート  | 配列をきっちり半分に分け、整列しながら合体させる。  |
| ソート  | クイックソート | ピボット（基準）を決め、大きい組と小さい組に分ける。 |
| 数学   | 行列の乗算   | 大きな行列を小さな行列に分解して計算する。      |

## 4. 疑似言語でのイメージ（再帰との関係）

分割統治法をプログラムで書くときは、「**再帰（自分自身を呼び出す）**」という手法がセットで使われます。

```
/* 分割統治法を行う関数 */
o divideAndConquer(Problem: p)
  if (p が十分に小さい)
    /* 答えを直接計算して返す */
    return 直接計算した答え
```

```
endif

// 分割
Problem:  $p1 \leftarrow p$  の前半部分
Problem:  $p2 \leftarrow p$  の後半部分

// 統治（再帰的に自分を呼ぶ）
Result:  $r1 \leftarrow \text{divideAndConquer}(p1)$ 
Result:  $r2 \leftarrow \text{divideAndConquer}(p2)$ 

// 統合
Result:  $\text{finalResult} \leftarrow r1$  と  $r2$  を組み合わせた結果

return finalResult
```

---

## 5. メリットとデメリット

- **メリット:**
    - 計算効率が非常に高い。
    - 並列処理（複数のCPUで分担して計算すること）と相性が良い。
  - **デメリット:**
    - 再帰を繰り返すため、メモリ（スタック領域）を多く消費することがある。
    - 問題の分割方法が悪いと、逆に効率が落ちる（クイックソートの最悪ケースなど）。
-